

The VPC-pro (joint) family: two roadblocks that need *both* techniques at once

Backdrop. *VPC-pro* (the joint family) collects the roadblocks that only fall to `value_profile_cmplog` — the fuzzer that runs input-to-state substitution (I2S, from `cmplog`) *and* the value-profile Hamming gradient (from `value_profile`) *together*. The defining property is *joint necessity*: `cmplog` alone blocks, `value_profile` alone blocks, and only the combination resolves (decisive shape LLWL / LLW_: the two single arms are losers L, only `vpc` is the winner W). Picture a door with *two* locks that must turn at the same instant: I2S holds one key, the gradient holds the other, and neither key alone opens it. That is what makes this family different from I2S-pro (one key, the literal) and VP-pro (one key, the gradient) — here *both* are load-bearing. The two categories below differ in *what the gradient adds*: *retain* the near-match so I2S can land an exact operand, or *grow* a larger structure. Counts are validated roadblocks (n).

How we prove jointness, not just “vpc won.” Each rule below is built so it can only fire when *both* contributions are visible: an I2S contribution (the exact literal/tag the gradient-only arm never places, so `value_profile` blocks) *and* a gradient contribution (the size growth or value closure the substitution-only arm never reaches, so `cmplog` blocks). A branch where one technique alone would have sufficed does not land here — it routes to I2S-pro or VP-pro instead.

1. Joint operand-reach: the exact literal is landed only by the combined arm

joint_value_distance_closure ($n = 18$)

The common case. The gate needs an *exact literal operand* — a SQL keyword (PRAGMA), a 4-byte table tag (OS/2, loca, CPAL) — and *neither* single technique gets that literal into its corpus. `value_profile` (gradient, no I2S) climbs *toward* the literal but, with no substitution stage, only ever produces near-misses ('SELE...' for PRAGMA) and never spells it. `cmplog` (I2S, no gradient) *could* substitute the literal, but with no distance feedback it cannot *retain and refine* a partially-correct seed long enough for the substitution to land — a half-right keyword earns no coverage credit and is dropped, so its corpus keeps only malformed fragments. Only `vpc` does both: the gradient *retains* the near-match, then I2S *writes* the exact bytes. **The measurable signature is operand-reach**: the exact literal is present in the joint corpus and absent from the single-technique corpora. (Note this is what the metric actually reads — byte-presence of the operand in each arm's seeds — *not* any tuning of downstream parser fields.)

```
// Category 1 -- joint operand-reach (simplified from sqlite3/16001,
    sqlite3NameFromToken)
// ONE exact literal, but it needs BOTH techniques to end up in the
    corpus at all:
tok = next_token(sql);                // the lexer matches the input
    against fixed keywords
if (memcmp(tok, "PRAGMA", 6) == 0) {  // GATE: the exact 6-byte keyword
    literal
```

```

    parse_pragma(sql);                                //  <-- resolving (false) side --
        blocker sqlite3/16001
    return RESOLVED;
}
return BLOCKED;
//  value_profile (VP only) : gradient keeps near-matches but can't
//  spell it -> 'SELE..'
//  cmplog          (I2S only): could write it, but with no gradient the
//  half-right seed
//
//  malformed fragment
//  vpc              (both)   : VP RETAINS the near-match, then I2S WRITES
//  'PRAGMA' -> lands it

```

Gate paraphrased from the analysis card (sqlite3 is the amalgamation; the keyword match feeds `sqlite3NameFromToken` at line 110953). The exact 6-byte operand 'PRAGMA' appears in the `vpc` corpus (residual 0.0) and is 5/6 bytes absent from the `cmplog` corpus (residual 0.83); the card states `cmplog` “cannot retain and refine inputs only partially closer to a valid keyword” — the retention the gradient supplies.

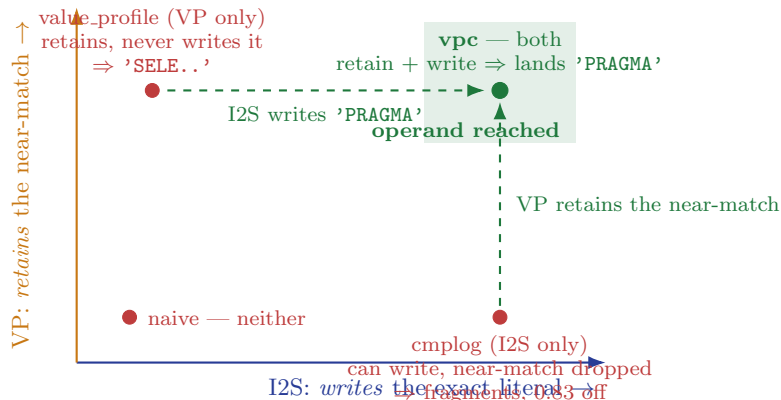
How we know it is really joint. Jointness itself is fixed by the decisive shape (LLW-/LLWL: `value_profile` alone blocks, `cmplog` alone blocks, only `vpc` resolves) — the rule’s job is to say *which* joint subtype and confirm it. First, `vpc` is *not* assembling a bigger input (`size_lift` < 1.3, so not the assembly-depth case of category 2 — the second key is a value, not structure). Second, the exact operand is *reached in the joint corpus but not the I2S-only corpus*: `value_distance_reached` slides the literal over each seed (per-byte Hamming ÷ width, closest seed per arm), and we require the `cmplog` corpus to sit a positive residual away while `vpc` lands it (`cmp_min_distance` > 0, `distance_closure_ratio` ≥ 0.5). The I2S half is corroborated by the third arm: `value_profile` (gradient, no substitution) carries the literal in 0% of seeds. *The metric reads operand byte-presence, i.e. retention × substitution — not field-tuning.*

Tool: `seed.size.and.tags + value.distance.reached`

Metric: `size_lift` = `vpc` / `cmplog` seed-size ratio (< 1.3 = not bigger ⇒ not assembly); `cmp_min_distance` = the I2S-only corpus’s closest Hamming residual to the *exact operand literal* (> 0 = the literal is absent from that corpus); `distance_closure_ratio` = fraction of that residual the joint corpus closes (1.0 = lands the operand)

Verify rule (the arbiter fires this, no LLM): `size_lift` < 1.3 AND `cmp_min_distance` > 0 AND `distance_closure_ratio` ≥ 0.5

Real example: `sqlite3/16001` (`sqlite3NameFromToken`, operand 'PRAGMA', width 6, well clear of the ≤ 2 chance-alignment flag) — `size_lift` = 0.31 (`vpc` is *smaller*, so not assembly); `cmp_min_distance` = 0.83 (`cmplog`’s closest seed is 5/6 bytes off — the keyword is genuinely absent) while `vpc_min_distance` = 0.0, so `distance_closure_ratio` = 1.0; `value_profile` produces near-keyword fragments but never the literal (I2S necessary).



Each single technique supplies one capability and stays outside the `operand-reached` corner: `cmplog` can write the literal but without the gradient its half-right seed is dropped, so the keyword never lands (0.83 residual); `value_profile` retains near-misses but can't write the exact bytes. Only `vpc` both retains the partial match *and* writes it, so `'PRAGMA'` is reached (residual $0.83 \rightarrow 0$). (sqlite3/16001, real metrics.)

Sub-cases. Of the 18 branches, 15 are the plain operand-reach form above; 2 (`joint_fixed_offset_literal_pre` harfbuzz/6858,7245) gate on a fixed-offset FOURCC where the blocking arm carries junk at the offset (confirmed by `signed_target_enrich` ≥ 1.0 instead, the operand-enrichment route); 1 (`joint_value_gradient` libxml2/7326, operand `'<![CDATA['`) is the same operand-reach on a longer string token scored purely on `value_distance_reached`. The mechanism — I2S writes the literal, the gradient retains the near-match — is identical; only the measurable proxy differs.

Caveat on the proxy. *Every* category-1 operand is itself a literal the I2S stage could in principle substitute (keywords / table tags: `PRAGMA`, `OS/2`, `loca`, `CPAL`, `CFF`, `<![CDATA['`). So the discriminating fact is never “`cmplog` can't write it” but “`cmplog`'s corpus doesn't *retain* a seed that lands it” — which is exactly the retention the gradient adds. The reading is clearest where the evidence says so outright (sqlite3/16001: `cmplog` “cannot retain and refine” near-keywords). It is least intuitive when the operand is a bare 4-byte tag the parser checks early, e.g. harfbuzz/5602 (operand `'OS/2'`, `cmp_min_distance` = 0.5): there a `vpc`-vs-`cmplog` distance gap is surprising precisely because `cmplog` has I2S, so that branch rests entirely on the retention reading rather than on any field-level distance. The `value_distance` number measures operand presence, not the downstream-field tuning an earlier draft implied.

2. Structural assembly depth under joint necessity `joint_assembly_depth` ($n = 9$)

Here the second key is not a precise value but *more structure / depth*. The gate sits behind interlocking fields. I2S places the exact tags/literals that keep the parser on a valid path — the gradient-only arm (`value_profile`) places *zero* of them and dies early — but a single I2S pass does not *grow* the input far enough, so `cmplog` assembles a partial structure and stalls. Only `vpc` climbs the gradient to build deeper. This shows up two ways: more loop *iterations* (`depth_reach`, 2 branches) or more bytes *assembled* (`seed_size_and_tags`, 7 branches). The clearest gate is an iteration one:

```
// harfbuzz src/hb-ot-post-table.hh : line 255 (verbatim source line)
// the resolving (false) side needs BOTH keys on one line: the exact
// version
// magic (I2S plants 0x00020000) AND glyph in bounds (glyph <
// glyphNameIndex->len),
// which the gradient reaches by driving the post-table glyph loop
// deeper per seed
if (version != 0x00020000 || glyph >= glyphNameIndex->len) // <--
    blocker (harfbuzz/5616)
    return false;
```

Verbatim source line at `post-table.hh:255`: one line carries *both* techniques — `0x00020000` is the I2S literal, and `glyph < glyphNameIndex->len` is the bound the gradient closes by iterating.

Stripped to its two keys (simplified for clarity — same gate, the loop the real one-liner hides made explicit):

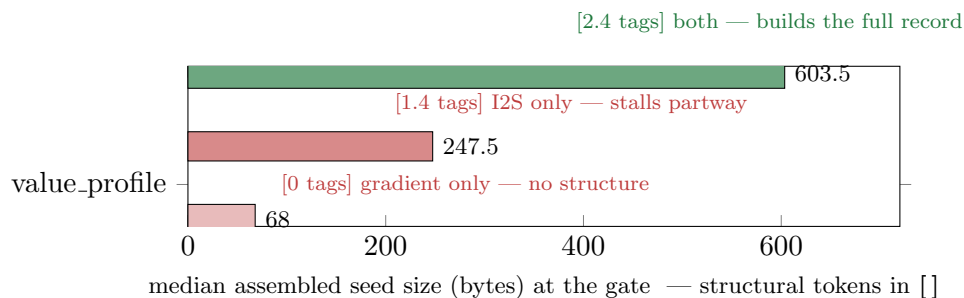
```
// Category 2A -- iteration DEPTH (simplified from harfbuzz/5616, post-
// table.hh:255)
if (version != 0x00020000) // KEY 1 (I2S): exact 4-byte version
    magic
    return BLOCKED; // value_profile never plants it ->
    blocks here
while (i < glyph) walk(i++); // KEY 2 (gradient): the index only
    comes in-bounds
// after the parser iterates this
// loop deep enough
if (glyph < glyphNameIndex->len) // cmplog enters the loop ONCE (depth
    ~1) and stalls;
    return RESOLVED; // only vpc re-enters + iterates
    deep (depth ~157x)
return BLOCKED;
```

How we know it is really joint. For an iteration gate we run each arm’s corpus through the instrumented coverage binary and count, per seed, how many times the blocker line executes (`depth_reach`). `value_profile` (gradient, no I2S) never places the version magic and never reaches the line; the discriminating signal is that `vpc` drives the line far deeper than the I2S-only `cmplog`.

Tool: `depth_reach`
Metric: `depth_lift` = `vpc` / `cmplog` per-seed executions of the blocker line; `winner_depth`, `loser_depth`; `winner_deeper` (bool)
Verify rule (the arbiter fires this, no LLM): `depth_lift` >= 1.5 AND `winner_deeper` == true
Real example: harfbuzz/5616 (post-table.hh:255) — `vpc` executes the gate line `winner_depth`= 157.2 times per seed against the `cmplog` arm’s `loser_depth`= 1.0 (`depth_lift`= 157.2): the gradient retains the near-miss inputs that let the corpus re-enter and iterate the gated loop, while I2S alone touches it once and stalls.

The other (and more common) form: byte growth. For 7 of the 9 branches the depth is raw size, scored by `seed.size_and_tags` (rule `vp_tags < 0.3 AND size_lift >= 1.3`). Real example harfbuzz/6814: the gradient-only arm assembles *no* table tags (`vp_tags`= 0, so I2S is necessary), while `vpc` grows the input 2.4× past the I2S-only arm (`size_lift`= 2.44):

```
// Category 2B -- assembly SIZE (simplified from harfbuzz/6814; cf. shape
// -normalize.cc:196)
if (!has_tag(font, "cmap") || !has_tag(font, "GSUB")) // KEY 1 (I2S):
    exact table tags
    return BLOCKED; // value_profile assembles 0 tags
    -> blocks (vp_tags=0)
record = build_full_table(font); // KEY 2 (gradient): the WHOLE record
    must be grown
if (record_complete(record)) // cmplog grows ~248 B (partial)
    and stalls;
    return RESOLVED; // only vpc grows it ~604 B (2.4x)
    -> resolves
return BLOCKED;
```



The gradient-only arm (`value_profile`) places no tags and stays tiny; I2S-only (`cmplog`) gets the tags but only a partial structure; `vpc` grows it 2.4× larger to reach the deep gate. (harfbuzz/6814, real metrics.)

At a glance

Category	<i>n</i>	Tool / metric	What the gradient adds
value_distance_closure	18	seed_size_and_tags + value_distance_reached / size_lift, distance_closure_ratio	<i>retain</i> the near-match so I2S lands the exact operand — reached in the joint corpus, absent from the I2S-only one (incl. 2 fixed-offset-presence, 1 string-token sub-cases)
assembly_depth	9	seed_size_and_tags / vp_tags, size_lift	grow a larger <i>structure</i> I2S alone never assembles (incl. 2 iteration-depth via depth_reach)
VPC-pro total	27	all share joint necessity: <code>cmplog</code> and <code>value_profile</code> each block alone; only <code>vpc</code> resolves	

Counts and example metrics are from `bench/dataset.jsonl`; verify rules are the deterministic arbiter rules (no LLM in the decision). The joint family is exclusive by its deciding pair $p \succ c \wedge p \succ v$ (`value_profile_cmplog` beats both single arms), so a branch resolvable by either technique alone never lands here. harfbuzz corpora live on the other server; the cited metrics were scored there and merged into the dataset.